

queue 정리

먼저 들어온 데이터를 먼저 처리하는 FIFO 방식의 C++ 표준 컨테이너 어댑터입니다.

사용처 (Where)

- 입력된 순서대로 데이터를 처리할 때
- 대기열이나 작업 순서를 관리할 때
- BFS 같은 탐색 알고리즘을 구현할 때

방법 (How to Use)

- queue 변수 선언
- push()로 뒤에 데이터 추가
- front()로 가장 앞의 데이터 확인
- pop()으로 가장 앞의 데이터 제거

기본 정보

- 필요 헤더: #include <queue>
- 기본 선언: queue<int> q;
- 처리 방식: FIFO (First In, First Out)

왜 써야 할까? (Why)

- FIFO 구조를 간단하게 구현할 수 있음
- 배열과 인덱스를 직접 관리할 필요가 없음
- 삽입 순서에 따른 처리가 명확함

주요 함수

함수/문법	의미
push(x)	뒤에 데이터 추가
pop()	가장 앞의 데이터 제거 (반환값 없음)
front()	가장 앞의 데이터 확인
back()	가장 뒤의 데이터 확인
size()	저장된 데이터 개수
empty()	비어 있는지 확인

기본 코드 예시

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;
    q.push(10); q.push(20); q.push(30);
    cout << q.front() << '\n';
    cout << q.back() << '\n';
    q.pop();
    cout << q.front() << '\n';
}
```

queue 비교와 응용

직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
front()	O(1)
back()	O(1)
push()	O(1)
pop()	O(1)
size(), empty()	O(1)

STL vs 직접 구현 vs C 스타일

구분	STL queue	직접 구현	C 스타일
표현력	FIFO 의도가 분명	구조를 직접 설계	배열과 인덱스 관리
코드 길이	짧고 표준적	필드와 함수 필요	초기화·경계 처리 필요
유지보수	안전하고 읽기 쉬움	설계가 늘수록 복잡	실수 가능성이 큼
추천 상황	일반적인 FIFO 처리	특수 규칙이 필요할 때	C API 연동이 필요할 때

응용: 입력 순서대로 출력하기

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;
    q.push(100); q.push(200); q.push(300);
    while (!q.empty()) {
        cout << q.front() << ' ';
        q.pop();
    }
}
```

처음 배울 때 자주 하는 실수

- 빈 queue에서 front() 또는 pop()을 호출함
- pop()이 제거한 값을 반환한다고 생각함: pop()은 반환값이 없는 void 함수
- queue의 중간 원소에 인덱스로 접근하려고 함
- 반복문에서 pop()을 빼먹어 무한 반복이 발생함

비슷한 항목과 차이

- stack은 마지막에 들어온 데이터를 먼저 처리하는 LIFO 구조
- deque는 앞과 뒤 양쪽에서 삽입과 삭제가 가능한 실제 컨테이너
- priority_queue는 입력 순서가 아니라 우선순위에 따라 처리
- queue는 FIFO 처리를 위한 제한된 인터페이스를 제공하는 컨테이너 어댑터

한 줄 정리: **queue**는 '먼저 들어온 데이터를 먼저 처리해야 할 때' 가장 먼저 떠올리면 좋은 STL입니다.

사용법 숙지를 위한 기본 예제

기본 예제 1. 첫 번째 손님 처리하기

이름을 들어온 순서대로 넣고, 맨 앞 손님을 처리합니다.

```
queue<string> waiting;
waiting.push("민수");
waiting.push("지수");

cout << waiting.front() << '\n';
waiting.pop();
cout << waiting.front() << '\n';
```

실행 결과: 민수 / 지수

기본 예제 2. 모든 숫자를 순서대로 꺼내기

empty()로 남은 값이 있는지 확인하면서 front()와 pop()을 함께 사용합니다.

```
queue<int> q;
q.push(1); q.push(2); q.push(3);

while (!q.empty()) {
    cout << q.front() << ' ';
    q.pop();
}
```

실행 결과: 1 2 3